QuillAudits

AUDIT REPORT

May 2026

For

AUMENT

Table of Content

Executive Summary

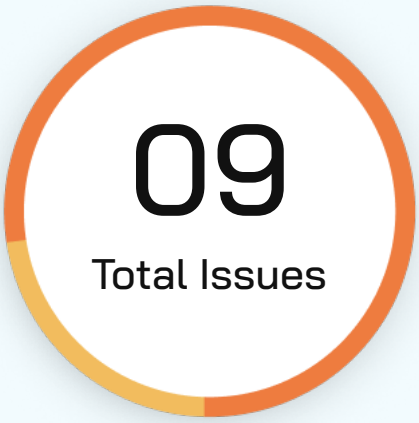
Project Name	Aument
Protocol Type	Escrow
Project URL	https://www.aument.com/
Overview	AUMENT Contracts represent a production-ready, gold-backed token system implemented as an ERC-20 smart contract suite. The system is specifically designed to support the tokenization of physical gold assets while ensuring full regulatory alignment through integrated KYC/AML compliance mechanisms, transaction monitoring, and controlled access via whitelisting. Governance and critical operations—including minting, burning, fee management, and contract upgrades—are secured through a multi-signature framework, ensuring robust oversight, transparency, and operational integrity. The architecture further incorporates advanced features for fee collection, redemption tracking, and upgradeability, supporting a scalable and compliant digital asset ecosystem. In addition, the AUMENT codebase includes a token-based loan facilitation framework, whereby token holders may utilize their tokens as collateral to obtain loans provided by regulated banking partners. Collateralized tokens are securely locked within a dedicated escrow smart contract for the duration of the loan term. Upon maturity, and subject to the fulfillment of predefined conditions, the escrow contract is deactivated and the tokens are released to the appropriate party (lender or borrower), ensuring a controlled and transparent collateral lifecycle.
Audit Scope	The scope of this Audit was to analyze the Aument Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/AUMENT-AG/AUME_2026
Branch	Main
Commit Hash	39524073dfed17ef6ac6d013bd5b5137fae814a0
Language	Solidity

Contracts in Scope	contracts/AumentWalletContract.sol contracts/EscrowContract.sol contracts/EscrowProxy.sol contracts/RevertMsg.sol contracts/Administrable.sol contracts/EscrowFactoryProxy.sol contracts/IERC20Aument.sol contracts/ERC20Aument.sol contracts/ProxyStorage.sol
Blockchain	EVM
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	17th April 2026 - 28th April 2026
Updated Code Received	30th April 2026
Review 2	4th May 2026
Fixed In	44d1722c1fd54a676827e444c9155f25d0909a60

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	0 (0.0%)
High	0 (0.0%)
Medium	7 (77.77%)
Low	2 (22.23%)
Informational	0 (0.0%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	0
Partially Resolved	0	0	0	0	0
Resolved	0	0	**7**	**2**	0

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Unsafe ERC20 withdrawal via raw transfer can cause stuck tokens	Medium	Resolved
2	Missing signature deadline in metaTransfer	Medium	Resolved
3	Missing chain binding in metaTransfer signature	Medium	Resolved
4	Insufficient oracle staleness validation in large-mint pricing	Medium	Resolved
5	ETH can be received by factory but no withdrawal path exists	Medium	Resolved
6	burnFrom reverts on blacklisted accounts when burn fee is active	Medium	Resolved
7	fulfillRedemption fee not reflected in stored request data	Medium	Resolved
8	Missing logic: RedemptionRequest.timestamp set but never used	Low	Resolved
9	initialize() incorrectly sets _mintFee from transferFee	Low	Resolved

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Medium Severity Issues

Unsafe ERC20 withdrawal via raw transfer can cause stuck tokens

Resolved

Path

AumentWalletContract.sol

Function Name

`withdrawAnyToken()`

Description

The wallet uses raw IERC20.transfer(recipient, amount) to withdraw arbitrary tokens.
Many widely used tokens (e.g., USDT-like implementations) are non-standard and do not return a boolean value on transfer.
In those cases, ABI decoding of the expected bool reverts, so withdrawals fail even when token balances exist and tokens can become stuck.

Impact

Supported treasury assets that do not return true/false on transfer can become non-withdrawable through this function.
This creates an operational DoS on token recovery/treasury management and can leave funds effectively stuck in the contract.

Likelihood

High in practice because non-standard ERC20 behavior is common on production chains.

Recommendation

Use OpenZeppelin SafeERC20 and replace raw transfer with `safeTransfer`.

Missing signature deadline in metaTransfer

Resolved

Path

ERC20Aument.sol

Function Name

`metaTransfer()`

Description

metaTransfe validates signature and nonce, but the signed payload has no deadline field.
A valid signature can be submitted at any future time until the nonce is consumed, allowing delayed execution long after user intent.

Impact

Old signatures remain executable indefinitely, creating stale-authorization risk.
A relayer or counterparty can intentionally delay submission and execute at a later, unfavorable time for the signer.

Likelihood

Medium in systems using off-chain signature relay flows, because delayed/batched relaying is common.

Recommendation

Include deadline in signed data and enforce block.timestamp <= deadline in metaTransfer.

Missing chain binding in metaTransfer signature

Resolved

Path

ERC20Aument.sol

Function Name

`metaTransfer()`

Description

The signed digest for metaTransfer does not include block.chainid.
Current replay protection uses per-signer nonce and address(this), which protects same-chain replay and cross-contract replay, but not cross-chain replay if the protocol is deployed on additional EVM chains in the future.

Impact

At present, this is Ethereum-only deployment.
If deployed later on other chains, a signature valid on one chain can be replayed on another chain where signer nonce state is still aligned, enabling unintended token transfers.

Likelihood

Low currently, but Medium in future multi-chain expansion scenarios.

Recommendation

Bind signatures to chain context by including block.chainid in the signed digest.
Keep existing per-signer nonce checks.

Insufficient oracle staleness validation in large-mint pricing

Resolved

Path

ERC20Aument.sol

Function Name

`_isLargeMintByUsd()`

Description

_isLargeMintByUsd only checks updatedAt == 0 after latestRoundData().
This does not enforce freshness in normal operation; stale but non-zero timestamps still pass.
As a result, old oracle prices can be used to classify mints as large/small and apply incorrect mint-fee behavior.

Impact

During oracle outages or delayed updates, stale price data may drive mint-fee decisions.
This can cause fee under-collection or over-collection relative to intended protocol policy.

Likelihood

Medium. Oracle update delays and heartbeat misses are realistic operational conditions.

Recommendation

Add an explicit freshness bound with a configurable threshold aligned to feed heartbeat and risk appetite.

ETH can be received by factory but no withdrawal path exists

Resolved

Path

EscrowFactoryProxy.sol

Function Name

`deployAndCloseEscrow() / receive()`

Description

EscrowFactoryProxy can receive ETH via its receive() function, and closeEscrow flow routes remaining ETH from escrow proxies back to the factory.
However, the factory contract does not expose any function to withdraw or forward accumulated ETH.
As a result, ETH sent to or returned to the factory can accumulate without an on-chain recovery mechanism.

Impact

ETH returned from escrow closures (or accidentally sent to the factory) can become operationally stuck in EscrowFactoryProxy.
This creates treasury fund lock risk and reduces recoverability of native assets.

Likelihood

Medium. The factory is explicitly designed to receive ETH, so accumulation is expected over time.

Recommendation

Add an onlyAdmin ETH withdrawal that can withdraw ETH to a specific address or msg.sender (admin).

burnFrom reverts on blacklisted accounts when burn fee is active

Resolved

Path

ERC20Aument.sol

Function Name

`burnFrom(address account, uint256 amount)`

Description

burnFrom is intended to burn tokens from any account, including blacklisted ones. However, when _burnFee > 0 and burnFeeCollector != address(0), the function first attempts to transfer the fee via: super._transfer(account, burnFeeCollector, fee);
This flows through _update() which checks: if (_blacklist[from]) revert SenderBlacklisted(from);
The entire burnFrom call reverts for blacklisted accounts.

This breaks a core compliance capability: the protocol cannot destroy tokens held by sanctioned accounts without first disabling the burn fee (extra transaction, race condition window) or temporarily un-blacklisting the account (defeats blacklisting purpose).

Impact

Medium. Core admin compliance function entirely blocked whenever burn fees are active.

Likelihood

Medium. Occurs unconditionally for any blacklisted account with active burn fee.

Recommendation

Avoid transferring from the blacklisted account and consider a different approach

fulfillRedemption fee not reflected in stored request data

Resolved

Path

ERC20Aument.sol

Function Name

`_isLargeMintByUsd()`

Description

When fulfillRedemption runs, in some case no burn fee and in other case burn fee is deducted but no update reflects this to the frontend, as well as the redemptionRequests(requestId).amount is never updated to reflect the net burned amount. The RedemptionFulfilled event only logs requestId and proofHash -- no amount. Off-chain gold delivery systems that read the stored amount see the pre-fee value, not what was actually burned. This creates an ambiguity: users could receive more gold than was burned on-chain, creating a reserve shortfall.

Impact

Potential discrepancy between on-chain state and off-chain gold delivery logic

Likelihood

Medium. Occurs on every fulfillRedemption call when _burnFee > 0..

Recommendation

Reflect the amount or emit it in the event e.g emit RedemptionFulfilled(requestId, proofHash, netAmount, fee);

Low Severity Issues

Missing logic: RedemptionRequest.timestamp set but never used

Resolved

Path
ERC20Aument.sol

Function Name
`initialize()`

Description
The timestamp field is populated on requestRedemption but never read by any contract function. No time-based expiry, dispute window, or timeout exists. Depending on the intented functionality this could be critical if its a core feature that was not implemented

Recommendation
Either add the missing logic if any was intended or remove the unused field to save gas.

initialize() incorrectly sets _mintFee from transferFee

Resolved

Path
ERC20Aument.sol

Function Name
`initialize()`

Description
In initialize(), the contract sets: _mintFee = transferFee. This couples mint fee to transfer fee at initialization time and prevents independent initial configuration of mint fee policy. The initializer should accept a dedicated mintFee parameter and validate it against the same fee cap.

Recommendation
Add mintFee as a separate initializer parameter and set `_mintFee = mintFee` after applying cap validation.

Formal Verification Specs

ERC20Aument.spec

- non-admin must not set transfer fee
- non-admin must not set burn fee
- non-admin must not set mint fee
- non-admin must not mint
- non-admin must not pause
- non-admin must not blacklist
- non-admin must not fulfill redemption
- non-admin must not cancel redemption
- non-admin must not upgrade implementation
- non-admin must not add to whitelist (validated in isolated whitelist access-control spec run)
- transfer fee must stay within max cap
- burn fee must stay within max cap
- mint fee must stay within max cap
- a redemption request must never be both fulfilled and cancelled
- when paused, non-admin transfer must revert
- when paused, admin transfer can succeed under constrained admin-to-admin assumptions
- blacklisted sender must be blocked
- blacklisted recipient must be blocked
- non-whitelisted recipient must be blocked when whitelist mode is enabled
- replayNonce must never decrease for any address
- double fulfill must revert
- fulfilling a cancelled request must revert
- double cancel must revert
- cancelling a fulfilled request must revert

- ✓ mint must increase total supply by the requested amount
- ✓ successful burn must decrease total supply (otherwise revert path is accepted)
- ✓ setting wallet address to non-zero updates it correctly
- ✓ setting wallet address to zero must revert

AumentWalletContract.spec

- ✓ withdrawing ETH to zero recipient must revert
- ✓ withdrawing ETH above contract balance must revert
- ✓ withdrawing token with zero token address must revert
- ✓ withdrawing token to zero recipient must revert
- ✓ proxy call with value to zero target must revert
- ✓ proxy call without value to zero target must revert

EscrowContract.spec

- ✓ only factory can close escrow; non-factory caller must revert
- ✓ closing paid escrow to zero user address must revert
- ✓ closeEscrow has a reachable revert path
- ✓ closeEscrow has a reachable success path

EscrowFactoryProxy.spec

- ✓ deployAndCloseEscrow has a reachable revert path
- ✓ deployAndCloseEscrow has a reachable success path

Functional Tests

ERC20Aument.sol

- initialize is called once successfully; second initialize call reverts
- initialize with transferFee > cap reverts; initialize with burnFee > cap reverts
- initialize grants DEFAULT_ADMIN to deployer and ADMIN/MINTER/UPGRADER to each whitelisted admin
- non-admin calling setTransferFee / setBurnFee/setMintFee / setGoldPriceFeed / setAumentWalletAddress reverts
- setTransferFee/setBurnFee/setMintFee with fee above cap reverts
- setTransferFeeCollector/setBurnFeeCollector/setMintFeeCollector with zero address reverts
- pause called by admin succeeds; pause called by non-admin reverts
- when paused, transfer by non-admin reverts
- when paused, transfer by admin succeeds
- transfer from blacklisted sender reverts; transfer to blacklisted recipient reverts
- with whitelist enabled, transfer to non-whitelisted non-admin non-exempt recipient reverts
- requestRedemption with amount=0 reverts
- requestRedemption by blacklisted account reverts
- requestRedemption with insufficient balance reverts
- fulfillRedemption on non-existent requestId reverts
- fulfillRedemption on already-fulfilled request reverts
- fulfillRedemption on cancelled request reverts
- cancelRedemption on non-existent requestId reverts
- cancelRedemption on already-cancelled request reverts
- cancelRedemption on fulfilled request reverts
- _authorizeUpgrade by admin without UPGRADER role reverts; with UPGRADER role succeeds
- initialize sets _mintFee = transferFee; mint fee cannot be independently initialized at deployment

- metaTransfer signature has no deadline, so old signatures remain valid until nonce is consumed
- metaTransfer hash omits block.chainid, so future multi-chain deployment introduces latent cross-chain replay risk
- _isLargeMintByUsd only checks updatedAt == 0; stale-but-nonzero oracle timestamps are accepted

AumentWalletContract.sol

- withdrawAnyToken by admin succeeds for compliant ERC20
- withdrawAnyToken by non-admin reverts
- withdrawAnyToken with zero tokenAddress reverts
- withdrawAnyToken with zero recipient reverts
- withdrawEther by admin succeeds when balance is enough
- withdrawEther by non-admin reverts
- withdrawEther with zero recipient reverts
- withdrawEther with amount > contract balance reverts
- proxyCallWithValue by admin succeeds when target has code and wallet has enough ETH
- proxyCallWithValue by non-admin reverts
- proxyCallWithValue with zero target reverts
- proxyCallWithoutValue by admin succeeds on valid target
- proxyCallWithoutValue by non-admin reverts
- proxyCallWithoutValue with zero target reverts
- withdrawAnyToken uses raw IERC20.transfer; USDT-like non-standard tokens can revert and become non-withdrawable

EscrowContract.sol

- closeEscrow called by non-factory sender reverts with NotFactory
- closeEscrow called by stored factory sender succeeds when downstream token transfer returns true
- closeEscrow with loanPaid=true routes escrow token balance to userAddress

- closeEscrow with loanPaid=false routes escrow token balance to Aument wallet address
- closeEscrow reverts with EscrowTransferFailed when token transfer returns false
- ETH forwarding to factory checks success and reverts with EtherTransferFailed on failure

EscrowContract.sol

- deployAndCloseEscrow by admin succeeds with valid bytecode/salt and emits CreatedEscrowContract + LoanRepaid
- deployAndCloseEscrow by non-admin reverts
- duplicate salt/bytecode deployment path causes create2 failure and reverts with Create2Failed
- factory receives ETH from escrow close flow through receive()
- factory can receive ETH but has no withdraw/forward function, so ETH remain stuck

EscrowProxy.sol

- constructor with zero implementation address reverts
- constructor with zero ERC20 address reverts
- constructor stores factory as _escrowFactoryAddress
- _implementation() returns stored implementation address

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

ERC20Aument

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ERC20PausableUpgradeable (OZ)	Base Contract	Overrides _update to block transfers when paused; admin bypass handled in ERC20Aument._update	Pause flag correctly blocks non-admin transfers	If _update override order is incorrect, pausing may silently not work
ERC20BurnableUpgradeable (OZ)	Base Contract	burn and burnFrom with allowance checks; ERC20Aument overrides both to inject fee logic	Correct burn implementation; allowance checked before burn	Override must call super carefully to avoid double-counting
AccessControlUpgradeable (OZ)	Base Contract	Role-based access: ADMIN_ROLE, MINTER_ROLE, UPGRADER_ROLE, DEFAULT_ADMIN_ROLE	DEFAULT_ADMIN_ROLE held by a secured multisig or timelock	Compromise of DEFAULT_ADMIN_ROLE allows granting arbitrary roles; ADMIN_ROLE holders can mint, burn, pause, and upgrade

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
UUPSUpgradeable (OZ)	Base Contract	Upgrade authorization gated by _authorizeUpgrade (requires ADMIN_ROLE + UPGRADER_ROLE)	Upgrade process correctly restricted; new implementation preserves storage layout	Incorrect upgrade could introduce vulnerabilities or corrupt storage; __gap slots must be maintained
Initializable (OZ)	Base Contract	Ensures initialize() is called exactly once through the proxy	initialize() called only once at proxy deployment	Direct call on implementation corrupts state; mitigated by _disableInitializers() in constructor
AggregatorV3Interface (Chainlink)	External Oracle	latestRoundData() to fetch gold/USD price; classifies mints as large (fee-applicable) or small	Feed is live, not stale, returns a valid positive price; address set by admin before minting	Stale price passes current check (only updatedAt==0 blocked, not age-based); oracle manipulation misclassifies mints; unset feed (address(0)) reverts blocking all large mints

AumentWalletContract

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Address (OZ)	Library	functionCall and functionCallWithValue for arbitrary proxy calls	Library correctly propagates reverts from target contracts	Compromised admin key allows total treasury drain via arbitrary calldata
IERC20 (OZ)	Interface	token.transfer() in withdrawAnyToken to send any ERC20 out of the treasury	Target token conforms to ERC20 interface	Non-standard ERC20 tokens (e.g. USDT) may not return bool; use safeTransfer
IERC20Aument	External Contract	hasRole(ADMIN_ROLE, msg.sender) on every admin function to verify caller role	ERC20Aument deployed and accessible; access control state is authoritative	If ERC20Aument is redeployed to new address, _erc20AumentContractAddress is not updatable (no setter), permanently breaking admin access

EscrowContract

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IERC20Aument	External Contract	balanceOf(address(this)) for escrowed balance; transfer() to user or wallet; getAumentWalletAddress() on default	ERC20Aument is live; wallet address is non-zero	If Aument wallet address changes between escrow creation and closeEscrow call, defaulted loan tokens go to the new wallet
ProxyStorage	Inherited Storage	Shared slots: _escrowImplementationAddress (slot 0), _erc20AumentContractAddress (slot 1), _escrowFactoryAddress (slot 2)	Layout identical between EscrowProxy and EscrowContract	Storage collision if layout order diverges during upgrades; diamond storage pattern recommended

EscrowFactoryProxy

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IERC20Aument	External Contract	hasRole(ADMIN_ROLE, msg.sender) to gate deployAndCloseEscrow	ERC20Aument live; admin role assignments correct	Same as AumentWalletContract: breaks permanently if ERC20Aument is redeployed
EscrowProxy	Deployed Contract	Deployed via CREATE2 using admin-supplied bytecode and salt; immediately called to close the position	Bytecode is correct EscrowProxy creation bytecode; salt is not reused	Bytecode not validated on-chain; malicious bytecode deploys unexpected contract; salt reuse causes Create2Failed revert

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

ERC20Aument

Contract Name	Function	Category
ERC20Aument	initialize(address[] adminWhitelist, uint256 transferFee, uint256 burnFee)	**What this function can do** Initialize the proxy: set token name/symbol, grant ADMIN/MINTER/UPGRADER to all adminWhitelist entries, set fee params, set fee collectors to deployer, whitelistEnabled=true. **What this function cannot/should not do** Be called more than once (initializer); set fees > 5*10^4 (5%). **Main invariant(s)** All adminWhitelist addresses receive all three operational roles; fees within bounds; whitelistEnabled=true. **Access level** External, initializer (once via proxy)
ERC20Aument	mint(address to, uint256 amount)	**What this function can do** Mint AUME; if classified as large mint by USD (Chainlink), deduct mint fee to mintFeeCollector first, mint net to recipient. **What this function cannot/should not do** Be called by non-ADMIN_ROLE; mint if goldPriceFeed unset when large-mint check triggers. **Main invariant(s)** fee + net = amount; if no fee, recipient gets full amount. **Access level** External, onlyAdmin

Contract Name	Function	Category
ERC20Aument	burn(uint256 amount)	**What this function can do** Burn caller tokens; deduct burn fee to burnFeeCollector; burn remainder. **What this function cannot/should not do** Be called by non-admin and non-AumentWallet (onlyWalletOrAdmin). **Main invariant(s)** Total supply decreases by (amount - fee); burnFeeCollector receives fee. **Access level** External, onlyWalletOrAdmin
ERC20Aument	burnFrom(address account, uint256 amount)	**What this function can do** Admin-forced burn from any account; deduct burn fee via transfer from account to burnFeeCollector; burn remainder. **What this function cannot/should not do** Succeed when account is blacklisted AND burnFee > 0 — fee _transfer reverts in _update (Issue 2). **Main invariant(s)** Caller must have ADMIN_ROLE; account must have sufficient balance and allowance. **Access level** External, onlyAdmin. KNOWN ISSUE: Reverts for blacklisted accounts when burn fee is active.

Contract Name	Function	Category
ERC20Aument	transfer(address recipient, uint256 amount) / transferFrom(address sender, address recipient, uint256 amount)	**What this function can do** Move tokens; apply transfer fee and compliance checks via _update hook. **What this function cannot/should not do** Transfer to/from blacklisted addresses; transfer to non-whitelisted recipient when whitelistEnabled (unless recipient is admin, fee collector, or address(this)). **Main invariant(s)** Caller balance decreases by amount; recipient receives (amount - fee); transferFeeCollector receives fee. **Access level** External, open to all token holders / approved spenders
ERC20Aument	metaTransfer(bytes memory signature, address to, uint256 value, uint256 nonce)	**What this function can do** Execute a transfer on behalf of an off-chain signer using ECDSA signature; increments signer replay nonce. **What this function cannot/should not do** Accept invalid signatures; accept wrong nonce (MetaTransferInvalidNonce). **Main invariant(s)** Signer nonce incremented exactly once; transfer follows same _update rules. **Access level** External, open to any relayer. KNOWN ISSUE: No deadline parameter; no chain ID in hash — signature valid indefinitely across forks.

Contract Name	Function	Category
ERC20Aument	requestRedemption(uint256 amount)	**What this function can do** Lock caller tokens in the contract (transfer to address(this)); record RedemptionRequest with unique requestId; emit RedemptionRequested. **What this function cannot/should not do** Accept zero amount; be called by blacklisted address; accept insufficient balance. **Main invariant(s)** Tokens move from caller to contract; unique requestId stored; collision detected and reverted. **Access level** External, open to all non-blacklisted token holders
ERC20Aument	fulfillRedemption(bytes32 requestId, string memory proofHash)	**What this function can do** Mark redemption fulfilled; transfer burn fee to burnFeeCollector; burn remaining escrowed tokens; emit RedemptionFulfilled. **What this function cannot/should not do** Fulfill already-fulfilled or cancelled requests; fulfill non-existent requests; be called by non-admin. **Main invariant(s)** fulfilled=true; fee + burned = original amount; contract balance decreases. **Access level** External, onlyAdmin. KNOWN ISSUE: request.amount not updated to net burned amount.

Contract Name	Function	Category
ERC20Aument	cancelRedemption(bytes32 requestId)	**What this function can do** Mark redemption cancelled; return full escrowed tokens to original requester. **What this function cannot/should not do** Cancel already-fulfilled or cancelled requests; be called by non-admin. **Main invariant(s)** cancelled=true; tokens returned to request.user; contract balance decreases. **Access level** External, onlyAdmin
ERC20Aument	pause() / unpause()	**What this function can do** Toggle paused state; when paused, _update blocks all non-ADMIN-caller transfers. **What this function cannot/should not do** Be called by non-admin. **Main invariant(s)** After pause(), all non-admin transfers revert. **Access level** External, onlyAdmin
ERC20Aument	blacklistAddress(address) / unblacklistAddress(address)	**What this function can do** Add/remove address from blacklist; blacklisted senders and recipients blocked in _update. **What this function cannot/should not do** Blacklist address(0). **Main invariant(s)** _blacklist[account] set/cleared atomically. **Access level** External, onlyAdmin

Contract Name	Function	Category
ERC20Aument	addWhitelistAddress(address) / removeWhitelistedAddress(address)	**What this function can do** Add/remove from transfer whitelist; removal uses O(1) swap-and-pop. **What this function cannot/should not do** Add already-whitelisted; remove non-whitelisted address (NotInWhitelist). **Main invariant(s)** _addressesWhitelistMapping, _addressesWhitelistArray, _whitelistIndex remain consistent. **Access level** External, onlyAdmin
ERC20Aument	enableWhitelist() / disableWhitelist()	**What this function can do** Toggle whitelistEnabled; when enabled, _update checks recipient whitelist. **What this function cannot/should not do** Be called by non-admin. **Main invariant(s)** whitelistEnabled reflects current mode. **Access level** External, onlyAdmin
ERC20Aument	setTransferFee / setBurnFee / setMintFee (uint256 fee)	**What this function can do** Update respective fee in basis points. **What this function cannot/should not do** Set fee > 5*10^precision (5%). **Main invariant(s)** Fees capped at 5*10^4. **Access level** External, onlyAdmin

Contract Name	Function	Category
ERC20Aument	setTransferFeeCollector / setBurnFeeCollector / setMintFeeCollector (address)	**What this function can do** Update fee collector addresses. **What this function cannot/should not do** Set to address(0). **Main invariant(s)** Collectors are non-zero after update. **Access level** External, onlyAdmin
ERC20Aument	setGoldPriceFeed (address feed)	**What this function can do** Set Chainlink AggregatorV3Interface for large-mint classification. **What this function cannot/should not do** Set to address(0). **Main invariant(s)** _goldPriceFeed is a valid non-zero oracle address. **Access level** External, onlyAdmin
ERC20Aument	setAumentWalletAddress(address)	**What this function can do** Update registered wallet address used by onlyWalletOrAdmin modifier. **What this function cannot/should not do** Set to address(0). **Main invariant(s)** _aumentWalletContractAddress is non-zero after update. **Access level** External, onlyAdmin

Contract Name	Function	Category
ERC20Aument	updateReserveAttestation(string memory attestationHash, uint256 goldOunces)	**What this function can do** Emit ReserveAttestationUpdated with IPFS hash and gold ounce count for proof-of-reserve. **What this function cannot/should not do** Accept empty attestationHash; be called by non-admin. No state change — emits only. **Main invariant(s)** Event emitted with block.timestamp. **Access level** External, onlyAdmin
ERC20Aument	_authorizeUpgrade(address)	**What this function can do** Authorize a UUPS implementation upgrade. **What this function cannot/should not do** Be called without both ADMIN_ROLE and UPGRADER_ROLE (double-gated: onlyAdmin modifier + explicit hasRole check). **Main invariant(s)** Caller must hold both roles. **Access level** Internal override, called by upgradeToAndCall

AumentWalletContract

Contract Name	Function	Category
AumentWalletContract	receive()	**What this function can do** Accept any ETH sent to the contract; emit ReserveAdded. **What this function cannot/should not do** Nothing — open to all senders. **Main invariant(s)** Contract ETH balance increases by msg.value. **Access level** External payable, unrestricted
AumentWalletContract	withdrawAnyToken(address tokenAddress, address recipient, uint256 amount)	**What this function can do** Transfer any ERC20 token held by the treasury to a recipient. **What this function cannot/should not do** Transfer to/from zero addresses; safely handle non-standard ERC20 (Issue 6). **Main invariant(s)** Treasury balance decreases; recipient balance increases. **Access level** External, onlyAdmin, nonReentrant. KNOWN ISSUE: Uses raw transfer() not safeTransfer().
AumentWalletContract	withdrawAnyToken(address tokenAddress, address recipient, uint256 amount)	**What this function can do** Transfer ETH from treasury to recipient. **What this function cannot/should not do** Send to address(0); send more than balance; be called by non-admin. **Main invariant(s)** ETH balance decreases by amount; EtherTransferFailed revert on failure. **Access level** External, onlyAdmin, nonReentrant

Contract Name	Function	Category
AumentWalletContract	proxyCallWithValue(bytes calldata _data, address targetAddress, uint256 value)	**What this function can do** Execute arbitrary external call with ETH value using wallet balance. **What this function cannot/should not do** Target address(0); be called by non-admin. **Main invariant(s)** Call reverts if target fails; reentrancy protected. **Access level** External, onlyAdmin, nonReentrant. NOTE: Highly privileged — compromised admin key allows total treasury drain.
AumentWalletContract	proxyCallWithoutValue(bytes calldata _data, address targetAddress)	**What this function can do** Execute arbitrary external call without ETH. **What this function cannot/should not do** Target address(0); be called by non-admin. **Main invariant(s)** Same as proxyCallWithValue but no ETH transferred. **Access level** External, onlyAdmin, nonReentrant

EscrowFactoryProxy

Contract Name	Function	Category
EscrowFactoryProxy	receive()	**What this function can do** Accept ETH returned from escrow closures. **What this function cannot/should not do** Nothing — open to all senders. **Main invariant(s)** Factory ETH balance increases. **Access level** External payable, unrestricted. KNOWN ISSUE: No withdrawal function — ETH permanently locked.
EscrowFactoryProxy	deployAndCloseEscrow(bytes memory bytecode, bytes32 salt, address userAddress, bool loanPaid)	**What this function can do** Deploy EscrowProxy via CREATE2 with provided bytecode and salt; call closeEscrow(userAddress, loanPaid) atomically; emit CreatedEscrowContract and LoanRepaid. **What this function cannot/should not do** Deploy if CREATE2 returns address(0) (salt reused / invalid bytecode); be called by non-admin. **Main invariant(s)** Escrow deployed and closed in the same transaction; tokens flow to user (loanPaid=true) or Aument wallet (false) **Access level** External, onlyAdmin

EscrowContract

Contract Name	Function	Category
EscrowContract	closeEscrow(address userAddress, bool loanPaid)	**What this function can do** Read AUME balance; if loanPaid=true transfer to userAddress, if false transfer to aumentToken.getAumentWalletAddress(); send ETH balance to factory (msg.sender). **What this function cannot/should not do** Be called by anyone other than _escrowFactoryAddress; transfer to address(0) if wallet is unset. **Main invariant(s)** Full AUME balance transferred; no tokens remain in contract after close. **Access level** External, restricted to _escrowFactoryAddress

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
AumentWalletContract	ERC20 (AUME + any token)	Protocol treasury: receives AUME and arbitrary ERC20 tokens; holds until admin withdrawal via withdrawAnyToken or proxy calls
EscrowProxy (per loan)	ERC20 (AUME)	Borrower collateral for a single loan position; held until factory calls closeEscrow; released to borrower (repaid) or swept to Aument wallet (defaulted)

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
ERC20Aument	mint()	— (new supply)	AUME to recipient; AUME fee to mintFeeCollector	ADMIN_ROLE	Chainlink oracle classifies large mints; stale price risk; goldPriceFeed must be set before minting
ERC20Aument	burn()	AUME from caller (AumentWallet or admin)	AUME destroyed; fee to burnFeeCollector	ADMIN_ROLE or AumentWalletContract	Fee deducted before burn; supply decreases by (amount − fee)
ERC20Aument	burnFrom()	AUME from target account (via allowance)	AUME destroyed; fee to burnFeeCollector	ADMIN_ROLE	Blocked for blacklisted accounts when burnFee > 0
ERC20Aument	transfer() / transferFrom()	AUME from sender	AUME to recipient (minus fee); fee to transferFeeCollector	Any token holder / approved spender	Blacklist and whitelist enforced in _update; fee deducted in same hook
ERC20Aument	metaTransfer()	AUME from signer (off-chain auth)	AUME to destination	Any relayer	No deadline; no chainId in hash; nonce prevents same-address replay only; cross-chain replay risk

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
ERC20Aument	requestRedemption()	AUME from caller (to address(this))	— (held in escrow at ERC20Aument)	Any non-blacklisted holder	Tokens locked until admin action; timestamp stored but unused; no user-side timeout
ERC20Aument	fulfillRedemption()	— (tokens already in contract)	AUME fee to burnFeeCollector; remaining AUME destroyed	ADMIN_ROLE	Net burned amount not reflected in request state or event; off-chain gold delivery may misread amount
ERC20Aument	cancelRedemption()	— (tokens already in contract)	Full original AUME returned to requester	ADMIN_ROLE	Full amount returned; no fee on cancel
AumentWallet Contract	receive()	ETH from any sender	–	Anyone	Correctly emits ReserveAdded
AumentWallet Contract	withdrawAnyToken()	–	ERC20 to recipient	ADMIN_ROLE	Raw transfer() used — non-standard ERC20 (e.g. USDT) may fail silently
AumentWallet Contract	withdrawEther()	–	ETH to recipient	ADMIN_ROLE	Balance check before send; reverts on ETH transfer failure

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
AumentWallet Contract	proxyCall WithValue( )	ETH from treasury	ETH to target; arbitrary state changes in target	ADMIN_ ROLE	No restriction on target or calldata; compromised admin = total treasury drain; reentrancy protected
EscrowProxy / EscrowContra ct	closeEscr ow(loanPai d=true)	— (AUME already in proxy)	Full AUME balance to borrower (userAddre ss)	EscrowF actoryPr oxy (admin-gated)	ETH balance forwarded to factory then permanently locked
EscrowProxy / EscrowContra ct	closeEscr ow(loanPai d=false)	— (AUME already in proxy)	Full AUME balance to AumentWa llet via getAumen tWalletAdd ress()	EscrowF actoryPr oxy (admin-gated)	Wallet address read at call time; if wallet changed since escrow creation, tokens go to new wallet
EscrowFactory Proxy	deployAnd CloseEscr ow()	— (triggers closeEscro w flows above)	Per closeEscr ow direction; ETH accumulat es in factory (permanen tly locked)	ADMIN_ ROLE	Bytecode not validated on-chain; salt reuse causes Create2Failed; ETH accumulates with no recovery path

Closing Summary

In this report, we assessed the security and overall code quality of the Aument V2 smart contracts according to the audit methodology and procedures described throughout this document. The audit included a combination of manual review, automated analysis, and testing of the in-scope contracts and associated business logic.

During the audit process, several findings of medium and low severity were identified. The Aument team responded promptly to all reported issues and successfully implemented the recommended remediations and improvements. Following the remediation review, all identified findings were verified as resolved.

Overall,the audited contracts demonstrate a solid security posture and adherence to current smart contract development best practices. The codebase is well-structured, follows established Solidity patterns, and incorporates appropriate security considerations across the reviewed components.

While no audit can guarantee the complete absence of vulnerabilities, the final reviewed version of the Aument V2 contracts significantly reduces the risk of common smart contract security issues within the audited scope. Based on the conducted assessment, no unresolved critical or high severity vulnerabilities were identified.

Security is an ongoing process, and we recommend continued monitoring, internal review procedures, and additional testing for future upgrades or protocol changes.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Ouraudit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

May 2026

For

AUMENT

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com